

Exploratory Analysis of Smart Sorting – Rotten Fruit Detection using Machine Learning

Project Documentation

1. Introduction

Project Title:

Smart Sorting – Rotten Fruit Detection using Machine Learning

Team Members:

G V Syam Prasad (Machine Learning & Backend)

Team Members (Frontend, Data Analysis, Deployment)

2. Project Overview

The goal of this project is to detect rotten fruits automatically using Machine Learning and image classification techniques. The system analyzes fruit images and classifies them as Fresh or Rotten to assist farmers, vendors, and distributors in sorting fruits efficiently.

This system helps reduce food wastage, improve fruit quality monitoring, and automate the fruit sorting process.

Features

- Automated fruit quality detection
- Image preprocessing and classification
- Fast prediction using deep learning models
- Web interface for uploading fruit images and viewing results

3. Architecture

Frontend

HTML, CSS, and JavaScript-based interface that allows users to upload fruit images and view prediction results.

Backend

Flask-based Python server that handles:

- Image upload requests
- Input validation

- Prediction logic
- Model inference

Model Layer

A trained **Deep Learning model (CNN / Transfer Learning)** used for image classification.

The model identifies whether the fruit is:

Fresh
or
Rotten

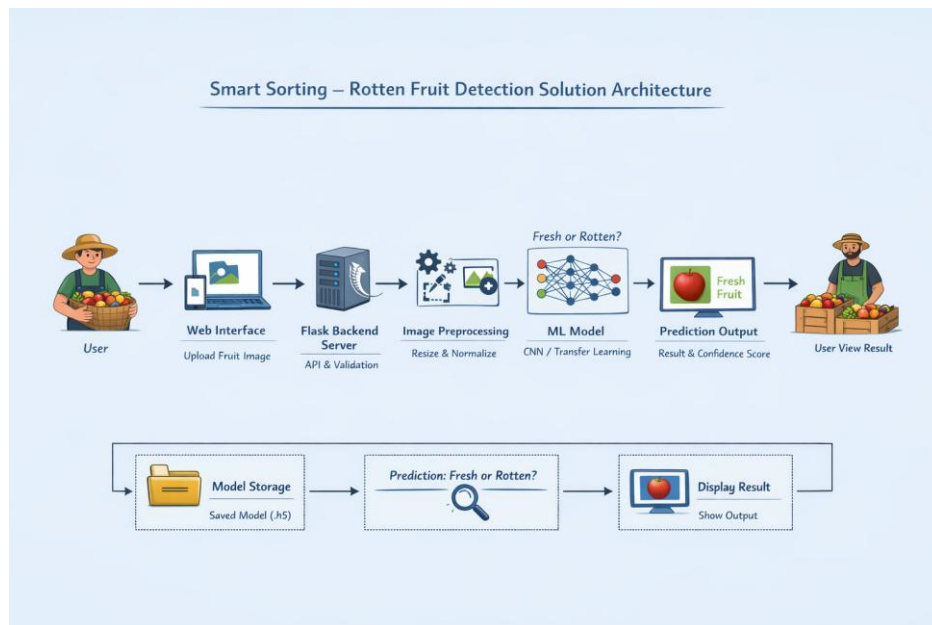
Model Storage

The trained model is saved as:

rotten_fruit_model.h5

This model is loaded by the backend server to perform predictions.

System Architecture Diagram



4. Setup Instructions

Prerequisites

- Python 3.x
- Flask
- TensorFlow / Keras
- NumPy
- OpenCV

Installation

Install dependencies using pip and run the backend application.

Example:

```
pip install flask tensorflow numpy opencv-python
```

5. Folder Structure

templates/	→ HTML interface files
static/	→ CSS, JS, Images
app.py	→ Flask backend server
model_training.ipynb	→ Model training notebook
rotten_vs_healthy	
l.h5	→ Trained deep learning model
dataset/	→ Fruit image dataset

6. Running the Application

Run the backend server:

```
python app.py
```

Open in browser:

```
http://localhost:5000
```

7. API Documentation

The system accepts **fruit image inputs** through the web interface and returns the **prediction result** indicating whether the fruit is fresh or rotten.

8. Authentication

Authentication is not implemented in the current version.

Future scope includes:

- Secure login system
- API authentication
- Token-based access control

9. User Interface

The user interface provides:

- Fruit image upload feature
- Prediction result display (Fresh / Rotten)
- Simple and user-friendly design

10. Testing

The machine learning model is evaluated using:

- Accuracy
- Precision
- Recall
- Confusion Matrix

The system was also tested for **prediction speed and interface responsiveness**

11. Screenshots / Demo

Screenshots of:

- Web interface
- Image upload page
- Prediction result page

should be included in this section.

12. Known Issues

- Prediction accuracy depends on dataset quality
- Performance may vary with poor image quality
- Model may require retraining for new fruit types

13. Future Enhancements

- Real-time fruit detection using camera input
- Mobile application for farmers and vendors
- Integration with automated fruit sorting machines

- Detection of multiple fruit diseases
- Cloud deployment for large-scale agricultural monitoring